

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Operating Systems (CO2018)

Assignment

“Simple Operating System”

Instructor(s): Nguyễn Phương Duy

Students: Lương Vĩnh Minh - 2212050
Vũ Đình Hoàn - 2211062
Nguyễn Hữu An Lợi - 2211942
Lê Võ Trọng Tín - 2213490

Group: L04 - Team 8

Ho Chi Minh City, November 2024



Contents

Danh sách thành viên và phân công	2
1 Giới thiệu	3
1.1 Mô tả đề tài	3
1.2 Source code	3
1.3 Mô tả các file process	4
2 Scheduler	7
2.1 Cơ sở lý thuyết	7
2.2 Trả lời câu hỏi	8
2.3 Hiện thực	8
2.3.1 Hiện thực queue.c	8
2.3.2 Hiện thực sched.c	9
2.3.3 Chỉnh sửa sched.h	11
2.3.4 Chỉnh sửa os.c	11
2.4 Kết quả	12
2.4.1 Testcase sched	12
2.4.2 Testcase sched_0	15
2.4.3 Testcase sched_1	17
3 Memory Management	22
3.1 Cơ sở lý thuyết	22
3.1.1 Bộ nhớ ảo (The Virtual Memory Mapping in Each Process)	22
3.1.2 Bộ nhớ vật lý của hệ thống (The System Physical Memory)	22
3.1.3 Sơ đồ dịch địa chỉ dựa trên phân trang	22
3.1.4 Phân đoạn bộ nhớ nhiều phần (Multiple Memory Segments)	23
3.1.5 Tóm tắt các triển khai theo định hướng phân trang	23
3.2 Trả lời câu hỏi	24
3.3 Hiện thực	27
3.3.1 Cấu trúc mã	27
3.3.2 Các hàm cần triển khai	27
3.3.3 Kết quả thực thi	32
4 Put It All Together	35
4.1 Cơ sở lý thuyết	35
4.2 Trả lời câu hỏi	35



Danh sách thành viên và phân công

STT	Tên	MSSV	Công việc	%
1	Vũ Đình Hoàn	2211062	- Hiện thực, giới thiệu	100%
2	Nguyễn Hữu An Lợi	2211942	- Sched, kết quả	100%
3	Lương Vĩnh Minh	2212050	- L ^A T _E X	10%
4	Lê Võ Trọng Tín	2213490	- Lý thuyết	100%

Table 1: Danh sách thành viên và phân công

1 Giới thiệu

1.1 Mô tả đề tài

Đây là bài báo cáo mô phỏng một hệ điều hành đơn giản. Mục tiêu của báo cáo là giải thích rõ những khái niệm cơ bản về scheduler, synchronization và memory management. Hình 1 minh họa cho kiến trúc tổng thể của hệ điều hành được triển khai. Tóm lại, hệ điều hành phải quản lý hai tài nguyên ảo là CPU và RAM thông qua hai thành phần chính:

- **Scheduler (và Dispatcher)**: Quyết định process nào được phép chạy trên CPU nào.
- **Virtual memory engine (VME)**: cô lập không gian bộ nhớ của các tiến trình khỏi các tiến trình khác. Bộ nhớ vật lý RAM được chia sẻ giữa nhiều tiến trình nhưng mỗi tiến trình đều không nhận biết được sự tồn tại của các tiến trình khác. Điều này được thực hiện bằng cách cấp cho mỗi tiến trình không gian bộ nhớ ảo riêng, VME sẽ ánh xạ và dịch địa chỉ ảo do các tiến trình cung cấp sang địa chỉ vật lý tương ứng.

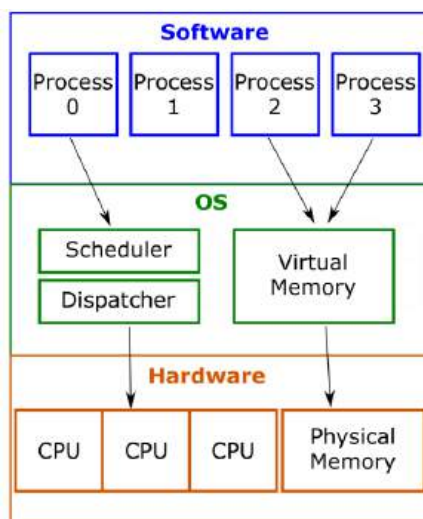


Figure 1: Tổng quan về mô hình chính trong bài tập lớn

Thông qua các module này, Hệ điều hành cho phép nhiều tiến trình được tạo ra bởi người dùng chia sẻ và sử dụng tài nguyên ảo của máy tính. Vì vậy, trong bài tập lớn này, nhóm sẽ tập trung hiện thực scheduler/dispatcher và virtual memory engine.

1.2 Source code

Source code được sắp xếp như sau:

- Header files
 - `timer.h`: Định nghĩa timer cho toàn bộ hệ thống.
 - `cpu.h`: Định nghĩa các hàm dùng cho hiện thực CPU ảo.
 - `queue.h`: Định nghĩa các hàm được sử dụng để triển khai hàng đợi chứa PCB của các process.

- `sched.h`: Định nghĩa các hàm dùng bởi scheduler.
- `mem.h`: Định nghĩa các hàm dùng bởi Virtual Memory Engine.
- `loader.h`: Định nghĩa các hàm được sử dụng bởi loader để nạp chương trình từ đĩa vào bộ nhớ.
- `common.h`: Định nghĩa các cấu trúc và hàm sử dụng chung trong hệ điều hành
- `bitopts.h`: Định nghĩa các thao tác trên dữ liệu bit
- `os-mm.h`, `mm.h`: Định nghĩa cấu trúc và dữ liệu cơ bản cho quản lý bộ nhớ dựa trên phân trang (Paging-based Memory Management).
- `os-cfg.h` (Tùy chọn): Định nghĩa các hằng số dùng để chuyển đổi cấu hình phần mềm.
- Source files
 - `timer.c`: Hiện thực timer
 - `cpu.c`: Hiện thực CPU ảo
 - `queue.c`: Hiện thực các thao tác trên hàng đợi ưu tiên (Priority Queue).
 - `paging.c`: Kiểm tra chức năng của Virtual Memory Engine.
 - `os.c`: Chứa hàm main để khởi động hệ điều hành.
 - `loader.c`: Hiện thực loader.
 - `sched.c`: Hiện thực scheduler.
 - `mem.c`: Triển khai phiên bản trước đây đã lỗi thời của RAM và Bộ nhớ Ảo.
 - `mm.c`, `mm-vm.c`, `mm-memphy.c`: Hiện thực Paging-based Memory Management.
- Makefile
- `input`: Thư mục chứa tập hợp các đầu vào để xác minh.
- `output`: “Các đầu ra mẫu của hệ thống.

1.3 Mô tả các file process

Để hiểu hơn về đề tài bài tập lớn, ta sẽ đi tìm hiểu ý nghĩa các file process nằm trong thư mục `input/proc`

`s0`

```
12 15
calc
calc
calc
calc
calc
calc
calc
calc
calc
```



```
calc  
calc  
calc  
calc  
calc  
calc  
calc
```

Process thực hiện 15 lệnh calc và có tổng thời gian thực thi (hay được gọi độ dài time slice): 15

p1s

```
1 10  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc  
calc
```

Process thực hiện 10 lệnh calc và có tổng thời gian thực thi (độ dài time slice): 10

p1s

```
1 7  
alloc 300 0  
alloc 100 1  
free 0  
alloc 100 2  
malloc 100 3  
write 102 1 20  
write 1 2 1000
```

- alloc 300 0: Yêu cầu cấp phát một vùng nhớ kích thước 300 bytes và gán ID là 0. Hệ thống sẽ tìm đủ số trang để ánh xạ cho vùng nhớ này và cập nhật Page Table tương ứng.
- alloc 100 1: Tương tự như trên, yêu cầu cấp phát một vùng nhớ kích thước 100 bytes và gán ID là 1.



- free 0: Giải phóng vùng nhớ có ID là 0. Thay vì thực sự thu hồi khung trang, vùng nhớ được đưa vào free list để sử dụng lại sau này.
- alloc 100 2: Cấp phát vùng nhớ kích thước 100 bytes với ID là 2.
- malloc 100 3: Tương tự alloc, nhưng cấp phát từ heap segment. Đây là vùng động trong bộ nhớ.
- write 102 1 20: Ghi dữ liệu kích thước 20 bytes tại địa chỉ 102 của vùng nhớ ID 1. Nếu địa chỉ nằm trên trang chưa được ánh xạ, hệ thống sẽ kích hoạt cơ chế page fault để ánh xạ trang này vào RAM.
- write 1 2 1000: Ghi dữ liệu kích thước 1000 bytes tại địa chỉ 1 của vùng nhớ ID 2. Vì dung lượng lớn, hệ thống có thể cần swap out các trang để tạo không gian trong RAM.

2 Scheduler

2.1 Cơ sở lý thuyết

Scheduler đóng vai trò quan trọng trong việc quyết định tác vụ hay chương trình nào được sử dụng CPU tại mỗi thời điểm. Do nhiều chương trình có thể chạy cùng lúc, hệ điều hành cần phải quản lý thời gian CPU để mọi chương trình đều có cơ hội thực thi.

Hệ điều hành được thiết kế để hoạt động với nhiều bộ xử lý. Nó sử dụng nhiều hàng đợi, gọi là **ready_queue** để xác định tiến trình nào sẽ được thực thi khi CPU sẵn sàng. Mỗi hàng đợi được liên kết với một giá trị ưu tiên cố định. Scheduler được thiết kế dựa trên giải thuật định thời MLQ (Multilevel queue) được sử dụng trong nhân Linux.

Đối với mỗi chương trình mới, loader sẽ tạo một tiến trình mới và gán một PCB mới cho tiến trình đó. loader sau đó đọc và sao chép nội dung của chương trình vào text segment của tiến trình mới. PCB của tiến trình được đẩy vào ready queue tương ứng, có cùng mức ưu tiên với giá trị ưu tiên của tiến trình đó. Sau đó, tiến trình sẽ chờ CPU. CPU chạy các tiến trình theo kiểu Round-robin. Mỗi tiến trình được phép chạy trong một khoảng thời gian nhất định (time slice). Sau đó, CPU sẽ buộc đưa tiến trình trở lại hàng đợi sẵn sàng ưu tiên liên kết với nó. CPU sau đó chọn một tiến trình khác từ ready queue và tiếp tục thực thi.

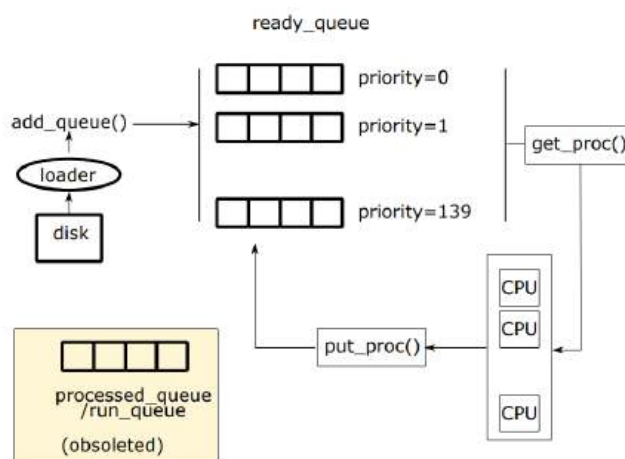


Figure 2: Hoạt động của scheduler

Trong hệ thống này, chúng tôi triển khai giải thuật định thời MLQ (Multilevel queue), phân chia các tiến trình thành nhiều hàng đợi riêng biệt được sắp xếp theo mức độ ưu tiên, và scheduler sẽ thực thi các tiến trình trong hàng đợi có mức độ ưu tiên cao nhất. Các thuật toán scheduling khác nhau có thể được sử dụng trong mỗi hàng đợi.

Đặc điểm của MLQ: bước duyệt qua ready queue là một số cố định được xác định dựa trên mức độ ưu tiên. Mỗi hàng đợi chỉ có một khoảng thời gian cố định để sử dụng CPU và khi khoảng thời gian này kết thúc, hệ thống phải chuyển tài nguyên sang tiến trình khác trong hàng đợi tiếp theo và để lại tiến trình còn lại để thực thi sau, ngay cả khi cần hoàn thành một vòng của hàng đợi sẵn sàng.

2.2 Trả lời câu hỏi

Câu hỏi Ưu điểm của việc sử dụng giải thuật MLQ sử dụng trong bài tập lớn so với các giải thuật định thời khác là gì?

So với các giải thuật định thời khác được học: FCFS, Shortest-Job-First, Shortest-remaining-Time-First, Round Robin Scheduling, Priority Scheduling with round-robin, giải thuật được triển khai trong bài tập lớn có những ưu điểm sau:

- **Phân bổ thời gian CPU tối ưu:** MLQ đảm bảo rằng các tiến trình có mức độ ưu tiên cao hơn được thực thi trước. Tuy nhiên, nhờ vào việc giới hạn thời gian cho mỗi mức độ ưu tiên, các tiến trình có mức độ ưu tiên thấp hơn vẫn được đảm bảo có đủ thời gian CPU, tránh được vấn đề đói tài nguyên.
- **Tính công bằng:** Giải thuật định thời MLQ đảm bảo phân bổ thời gian CPU công bằng cho các process khác nhau, dựa trên mức độ ưu tiên và yêu cầu của chúng.
- **Tính linh hoạt:** Giải thuật MLQ có thể được tùy chỉnh để đáp ứng các yêu cầu cụ thể của các process khác nhau. Bằng cách sửa đổi priority theo yêu cầu của người dùng và các hệ thống khác nhau.
- **Tối ưu tài nguyên tốt hơn:** Giải thuật định thời MLQ bảo sử dụng tài nguyên tốt hơn bằng cách thêm các process vào các hàng đợi theo thứ tự ưu tiên riêng biệt, quản lý hiệu quả.

2.3 Hiện thực

2.3.1 Hiện thực queue.c

Hàm: enqueue

Chức năng: Đưa process mới vào ready queue

Code:

```
1 void enqueue(struct queue_t *q, struct pcb_t *proc) {
2     /* TODO: put a new process to queue [q] */
3     if(q->size < MAX_QUEUE_SIZE){
4         q->proc[q->size] = proc;
5         q->size++;
6     }
7 }
```

Hàm: dequeue

Chức năng: Trả về process có mức độ ưu tiên cao nhất trong ready queue và lấy nó ra

Code:

```
1 struct pcb_t *dequeue(struct queue_t *q) {
2     /* TODO: return a PCB whose priority is the highest
3      * in the queue [q] and remember to remove it from q
4      */
5     if (empty(q)) return NULL;
6     struct pcb_t *proc = NULL;
7     #ifdef MULTI_SCHED
8         if (empty(q)) {
9             return proc;
10        }
```

```
10     }
11     else {
12         proc = q->proc[0];
13         for (int i = 0; i < q->size - 1; i++) {
14             q->proc[i] = q->proc[i + 1];
15         }
16         q->proc[q->size - 1] = NULL;
17         q->size--;
18     }
19     return proc;
20 #else
21     int highest_priority = MAX_PRIO * 2;
22     int index = -1;
23     for (int i = 0; i < q->size; i++) {
24         if (highest_priority > q->proc[i]->priority) {
25             highest_priority = q->proc[i]->priority;
26             index = i;
27         }
28     }
29     if (index == -1) return NULL;
30     else {
31         proc = q->proc[index];
32         for (int i = index; i < q->size - 1; i++) {
33             q->proc[i] = q->proc[i + 1];
34         }
35         q->size--;
36         q->proc[q->size] = NULL;
37     }
38 #endif
39     return proc;
40 }
```

2.3.2 Hiện thực sched.c

Hàm: get_mlq_proc

Chức năng: Chức năng: do ta thực hiện MLQ theo cơ chế slot nên ready queue với độ ưu tiên cao hơn sẽ có số slot nhiều hơn, với slot là số lượt cố định để sử dụng CPU cho các process ở ready queue có priority tương ứng và khi sử dụng hết, hệ thống phải thay đổi tài nguyên sang process khác. $\text{slot} = \text{max_prior} - \text{current_priority}$. Với max_prior là biến global được nhóm tính toán lại bằng giá trị priority lớn nhất (ở file os.c) của các process để hạn chế hiện tượng starvation.

Code:

```
1 static struct queue_t ready_queue;
2 static struct queue_t run_queue;
3 static pthread_mutex_t queue_lock;
4
5 #ifdef MLQ_SCHED
6 static struct queue_t mlq_ready_queue[MAX_PRIO];
7 static int max_prior = 140;
8 #endif
9
```

```
10 #ifndef MLQ_SCHED
11 /*
12  * Stateful design for routine calling
13  * based on the priority and our MLQ policy
14  * We implement stateful here using transition technique
15  * State representation prio = 0 .. MAX_PRIO, curr_slot = 0..(MAX_PRIO -
    prio)
16  */
17 struct pcb_t * get_mlg_proc(void) {
18     struct pcb_t * proc = NULL;
19     /* TODO: get a process from PRIORITY queue.
20      * Remember to use lock to protect the queue.
21      */
22     static int current_priority = 0;
23     static int slot = -1;
24
25     if (slot == -1) {
26         pthread_mutex_lock(&queue_lock);
27         slot = max_prior;
28         pthread_mutex_unlock(&queue_lock);
29     }
30
31     int turn = 0;
32     while (turn < 2) {
33         if (current_priority > max_prior) {
34             turn++;
35             pthread_mutex_lock(&queue_lock);
36             current_priority = 0;
37             slot = max_prior;
38             pthread_mutex_unlock(&queue_lock);
39         }
40     }
41
42     if (empty(&mlq_ready_queue[current_priority])) {
43         pthread_mutex_lock(&queue_lock);
44         current_priority++;
45         slot = max_prior - current_priority;
46         pthread_mutex_unlock(&queue_lock);
47     }
48     else {
49         pthread_mutex_lock(&queue_lock);
50         proc = dequeue(&mlq_ready_queue[current_priority]);
51         slot--;
52         pthread_mutex_unlock(&queue_lock);
53         if (slot <= 0) {
54             pthread_mutex_lock(&queue_lock);
55             current_priority++;
56             slot = max_prior - current_priority;
57             pthread_mutex_unlock(&queue_lock);
58         }
59         break;
60     }
61     return proc;
```

62 }

Hàm: `init_scheduler`

Thay đổi: Hàm này đã được nhóm chỉnh sửa để nhận số đầu vào `int max` từ tệp `os.c`, là giá trị `max_prior` được nhóm tính toán và đã được đề cập phía trên. Nếu là `MLQ_SCHED`, hàm sẽ gán giá trị `max` vào biến toàn cục `max_prior` để hàm `get_mlg_proc` có thể sử dụng.

Code:

```
1 void init_scheduler(int max) {
2     #ifdef MLQ_SCHED
3         int i;
4         for (i = 0; i < MAX_PRIO; i++) {
5             mlq_ready_queue[i].size = 0;
6         }
7         max_prior = max;
8     #endif
9     ready_queue.size = 0;
10    run_queue.size = 0;
11    pthread_mutex_init(&queue_lock, NULL);
12 }
```

2.3.3 Chỉnh sửa sched.h

Hàm: `init_scheduler`

Thay đổi: Chỉnh sửa khai báo để nhận thêm tham số đầu vào `int max`.

Code:

```
1 void init_scheduler(int max);
```

2.3.4 Chỉnh sửa os.c

Hàm: `read_config`

Thay đổi: Thêm biến global `max_prior` là giá trị priority lớn nhất của các process như đã đề cập phía trên. Biến này được tính trong vòng lặp qua các process, sau khi tìm được giá trị lớn nhất thì cộng thêm 1 và gán vào biến `max_prior`.

Code:

```
1 static int max_prior = 140;
2
3 static void read_config(const char * path) {
4     ...
5     #ifdef MLQ_SCHED
6         ld_processes.prio = (unsigned long*) malloc(sizeof(unsigned long) *
7             num_processes);
8     #endif
9     int i;
10    int max = 0;
11    for (i = 0; i < num_processes; i++) {
12        ld_processes.path[i] = (char*)malloc(sizeof(char) * 100);
13        ld_processes.path[i][0] = '\0';
14        strcat(ld_processes.path[i], "input/proc/");
15        char proc[100];
```

```
15
16 #ifndef MLQ_SCHED
17     fscanf(file, "%lu %s %lu\n", &ld_processes.start_time[i], proc, &
18         ld_processes.prio[i]);
19     if (ld_processes.prio[i] > max) {
20         max = ld_processes.prio[i];
21     }
22 #else
23     fscanf(file, "%lu %s\n", &ld_processes.start_time[i], proc);
24 #endif
25     strcat(ld_processes.path[i], proc);
26     max_prior = max + 1;
27 }
```

Hàm: main

Thay đổi: Truyền biến global max_prior sau khi đã tính toán vào hàm init_scheduler.

Code:

```
1 static int max_prior = 140;
2
3 int main(int argc, char *argv[]) {
4     ...
5     /* Init scheduler */
6     init_scheduler(max_prior);
7     ...
8 }
```

2.4 Kết quả

2.4.1 Testcase sched

Input:

```
4 2 3
1048576 16777216 0 0 0
0 p1s
1 p2s
2 p3s
```

Output:

Nhận xét:

Time slice: 4 – Mỗi process được thực thi trong 4 đơn vị thời gian trước khi bị chuyển ra khỏi CPU.

Số lượng CPU: 2 – Có hai CPU (CPU 0 và CPU 1) trong hệ thống.

Số lượng process: 3 (PID 1, PID 2, PID 3).

Với priority của p1s là 1, p2s và p3s là 0.

1048576: Kích thước RAM (1 MB).

16777216: Kích thước SWAP (16 MB).

0 0 0: Không có các thiết bị SWAP bổ sung.

```
Time slot 0
ld_routine
  Loaded a process at input/proc/p1s, PID: 1 PRIQ: 1
  CPU 0: Dispatched process 1
Time slot 1
  Loaded a process at input/proc/p1s, PID: 2 PRIQ: 0
  CPU 1: Dispatched process 2
Time slot 2
  Loaded a process at input/proc/p1s, PID: 3 PRIQ: 0
Time slot 3
Time slot 4
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 3
Time slot 5
  CPU 1: Put process 2 to run queue
  CPU 1: Dispatched process 2
Time slot 6
Time slot 7
Time slot 8
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 3
Time slot 9
  CPU 1: Put process 2 to run queue
  CPU 1: Dispatched process 2
Time slot 10
Time slot 11
  CPU 1: Processed 2 has finished
  CPU 1: Dispatched process 1
Time slot 12
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 3
Time slot 13
Time slot 14
  CPU 0: Processed 3 has finished
  CPU 0 stopped
Time slot 15
  CPU 1: Put process 1 to run queue
  CPU 1: Dispatched process 1
Time slot 16
Time slot 17
  CPU 1: Processed 1 has finished
  CPU 1 stopped
```

Figure 3: Kết quả testcase sched

Phân tích kết quả:

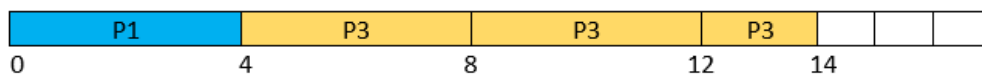
- Tại time slot 0, process 1 được gán PID = 1, load vào hàng đợi với độ ưu tiên là 1. CPU 0 duyệt hàng đợi và bắt đầu xử lý PID 1.
- Tại time slot 1, do lúc này hàng đợi đang trống nên process 2 (PID = 2) được load vào với độ ưu tiên là 0 (cao hơn process 1) đồng thời, CPU 1 bắt đầu xử lý PID 2.
- Tương tự tại time slot 2, process 3 (PID = 3) được load vào hàng đợi với độ ưu tiên là 0.
- Tại time slot 3, do 2 CPU đã được sử dụng và chưa hết time slice nên process 3 phải chờ.
- Tại time slot 4 timeslice của PID 1 trên CPU 0 đã hết, PID 1 được đưa vào hàng đợi. CPU 0 chuyển sang thực thi process PID 3 (priority = 0).
- Tại time slot 8 timeslice của PID 3 trên CPU 0 đã hết, PID 3 được đưa vào hàng đợi. CPU 0 duyệt hàng đợi và tiếp tục lấy ra PID 3 để thực thi (do PID 3 có priority = 0 cao hơn PID 1 có priority = 1).
- Time slot 9 time slice của PID 2 trên CPU 1 đã hết, nhưng PID 2 lại được tiếp tục xử lý (vì hàng đợi không còn process ưu tiên cao hơn).
- Time slot 11 PID 2 đã hoàn tất (đã chạy hết 10 time slot). CPU 1 tiếp tục thực thi PID 1.
- Time slot 14 PID 3 hoàn tất, CPU 0 không còn process nào để xử lý và dừng lại.



- Time slot 15 CPU 1 đưa PID 1 vào hàng đợi và tiếp tục xử lý nó.
- Time slot 17 PID 1 đã hoàn tất. CPU 1 không còn process nào để xử lý và dừng lại.

Biểu đồ Gantt:

CPU 0:



CPU 1:



2.4.2 Testcase sched_0

Input:

```
2 1 2
1048576 16777216 0 0 0
0 s0
4 s1
```

Nhận xét:

Hệ thống chỉ có 1 CPU (CPU 0) và xử lý lần lượt các process theo thời gian. Với time slice là 2, có priority tối đa là 5.

Biểu đồ Gantt:

CPU 0:



Phân tích kết quả:

- Time slot 0: s0 được tải vào hệ thống và gán PID = 1, có priority = 4 (ưu tiên thấp). CPU 0 ngay lập tức bắt đầu thực thi PID 1.
- Time slot 1: s0 thứ hai được tải vào hệ thống, gán PID = 2, có priority = 0 (ưu tiên cao nhất). PID 2 được đưa vào hàng đợi sẵn sàng (ready queue) do CPU 0 đang bận xử lý PID 1.
- Time slot 2: CPU 0 đã xử lý hết time slice của PID 1 và đưa nó vào hàng đợi. Do PID 2 có priority cao hơn PID 1, CPU 0 chuyển sang xử lý PID 2.
- Time slot 4: Time slice của PID 2 kết thúc nhưng do không có process nào khác ưu tiên cao hơn, PID 2 tiếp tục được xử lý.
- Time slot 17: Do PID 2 có priority = 0 (cao nhất) nên liên tục được xử lý PID 2 đã hoàn tất. CPU 0 chuyển sang xử lý PID 1 từ hàng đợi.
- Time slot 30: tiếp tục xử lý PID 1 cho đến khi hoàn tất. Không còn process nào để xử lý, CPU 0 dừng hoạt động.

Output:

```
Time slot 0
ld_routine
    Loaded a process at input/proc/s0, PID: 1 PRI0: 4
    CPU 0: Dispatched process 1
Time slot 1
    Loaded a process at input/proc/s0, PID: 2 PRI0: 0
Time slot 2
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 2
Time slot 3
Time slot 4
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 5
Time slot 6
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 7
Time slot 8
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 9
Time slot 10
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2

Time slot 11
Time slot 12
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 13
Time slot 14
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 15
Time slot 16
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 2
Time slot 17
    CPU 0: Processed 2 has finished
    CPU 0: Dispatched process 1
Time slot 18
Time slot 19
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 20
Time slot 21
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 22
Time slot 23

Time slot 23
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 24
Time slot 25
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 26
Time slot 27
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 28
Time slot 29
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 30
    CPU 0: Processed 1 has finished
    CPU 0 stopped
```

Figure 4: Kết quả testcase sched_0

2.4.3 Testcase sched_1

Input:

```
2 1 4
1048576 16777216 0 0 0
0 s0
4 s1
6 s2
7 s3
```

Nhận xét:

Time slice: 2

Số lượng CPU: 1 (CPU 0) trong hệ thống.

Số lượng process: 4.

Output:

```
Time slot 0
ld_routine
    Loaded a process at input/proc/s0, PID: 1 PRI0: 4
    CPU 0: Dispatched process 1
Time slot 1
    Loaded a process at input/proc/s0, PID: 2 PRI0: 0
Time slot 2
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 2
    Loaded a process at input/proc/s0, PID: 3 PRI0: 0
Time slot 3
    Loaded a process at input/proc/s0, PID: 4 PRI0: 0
Time slot 4
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
Time slot 5
Time slot 6
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 7
Time slot 8
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 9
Time slot 10
```

```
CPU 0: Put process 2 to run queue
CPU 0: Dispatched process 3
Time slot 11
Time slot 12
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 13
Time slot 14
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 15
Time slot 16
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
Time slot 17
Time slot 18
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 19
Time slot 20
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 21
Time slot 22
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
```

```
Time slot 23
Time slot 24
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 25
Time slot 26
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 27
Time slot 28
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
Time slot 29
Time slot 30
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 31
Time slot 32
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 33
Time slot 34
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
```

```
Time slot 46
    CPU 0: Processed 3 has finished
    CPU 0: Dispatched process 4
Time slot 47
    CPU 0: Processed 4 has finished
    CPU 0: Dispatched process 1
Time slot 48
Time slot 49
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 50
Time slot 51
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 52
Time slot 53
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 54
Time slot 55
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 56
```

```
Time slot 35
Time slot 36
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 37
Time slot 38
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 39
Time slot 40
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 3
Time slot 41
Time slot 42
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 4
Time slot 43
Time slot 44
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 2
Time slot 45
    CPU 0: Processed 2 has finished
    CPU 0: Dispatched process 3
Time slot 46
```

```
Time slot 57
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 58
Time slot 59
    CPU 0: Put process 1 to run queue
    CPU 0: Dispatched process 1
Time slot 60
    CPU 0: Processed 1 has finished
    CPU 0 stopped
```

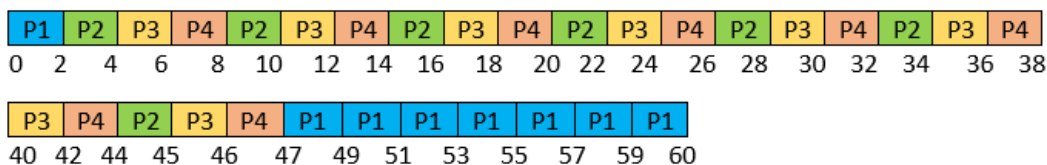
Figure 5: Kết quả testcase sched_1

Phân tích kết quả:

- Time slot 0: s0 được tải vào hệ thống, có PID = 1, có độ ưu tiên (PRIO) = 4 (ưu tiên thấp nhất). CPU0 gán PID 1 vào lịch
- Time slot 1: Tải tiến trình mới có PID = 2 và độ ưu tiên = 0 (Độ ưu tiên cao) nhưng chưa có hoạt động nào từ CPU.
- Time slot 2: CPU đưa tiến trình 1 vào hàng đợi trong khi bắt đầu thực hiện tiến trình 2. s0 được tải lên từ hệ thống với PID = 3 và độ ưu tiên = 0.
- Time slot 3: s0 được tải lên từ hệ thống với PID = 4 và PRIO = 0.
- Time slot 4-6: CPU xử lý từng tiến trình từ 1 đến 4 và đưa chúng vào hàng đợi khi hết thời gian xử lý.
- Time slot 45: PID 2 hoàn tất và CPU 0 chuyển sang xử lý PID 3.
- Time slot 46: PID 3 hoàn tất và CPU 0 chuyển sang xử lý PID 4.
- Time slot 47: PID 4 hoàn tất và CPU 0 chuyển sang xử lý PID 1.
- Time slot 60: PID 1 hoàn tất và CPU 0 dừng lại.

Biểu đồ Gantt:

CPU 0:



Input có priority:

```
2 1 4
1048576 16777216 0 0 0
0 s0 4
4 s1 0
6 s2 0
7 s3 0
```

Nhận xét:

Nếu input có priority

- 0 s0: process s0 xuất hiện tại time slot 0.
- 4 s1: process s1 xuất hiện tại time slot 4.
- 6 s2: process s2 xuất hiện tại time slot 6.
- 7 s3: process s3 xuất hiện tại time slot 7.

Phân tích kết quả:

```

Time slot 0
ld_routine
  Loaded a process at input/proc/s0, PID: 1 PRIO: 4
Time slot 1
  CPU 0: Dispatched process 1
Time slot 2
Time slot 3
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 4
  Loaded a process at input/proc/s1, PID: 2 PRIO: 0
Time slot 5
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 2
Time slot 6
  Loaded a process at input/proc/s2, PID: 3 PRIO: 0
Time slot 7
  CPU 0: Put process 2 to run queue
  CPU 0: Dispatched process 3
  Loaded a process at input/proc/s3, PID: 4 PRIO: 0
Time slot 8
Time slot 9
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 4
Time slot 10

```

```

Time slot 23
Time slot 24
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 2
Time slot 25
Time slot 26
  CPU 0: Put process 2 to run queue
  CPU 0: Dispatched process 3
Time slot 27
Time slot 28
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 2
Time slot 29
  CPU 0: Processed 2 has finished
  CPU 0: Dispatched process 3
Time slot 30
Time slot 31
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 3
Time slot 32
Time slot 33
  CPU 0: Put process 3 to run queue
  CPU 0: Dispatched process 3
Time slot 34
Time slot 35
  CPU 0: Processed 3 has finished

```

```

Time slot 11
  CPU 0: Put process 4 to run queue
  CPU 0: Dispatched process 4
Time slot 12
Time slot 13
  CPU 0: Put process 4 to run queue
  CPU 0: Dispatched process 4
Time slot 14
Time slot 15
  CPU 0: Put process 4 to run queue
  CPU 0: Dispatched process 4
Time slot 16
Time slot 17
  CPU 0: Put process 4 to run queue
  CPU 0: Dispatched process 4
Time slot 18
Time slot 19
  CPU 0: Put process 4 to run queue
  CPU 0: Dispatched process 4
Time slot 20
  CPU 0: Processed 4 has finished
  CPU 0: Dispatched process 2
Time slot 21
Time slot 22
  CPU 0: Put process 2 to run queue
  CPU 0: Dispatched process 3

```

```

  CPU 0: Dispatched process 1
Time slot 36
Time slot 37
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 38
Time slot 39
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 40
Time slot 41
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 42
Time slot 43
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 44
Time slot 45
  CPU 0: Put process 1 to run queue
  CPU 0: Dispatched process 1
Time slot 46
  CPU 0: Processed 1 has finished
  CPU 0 stopped

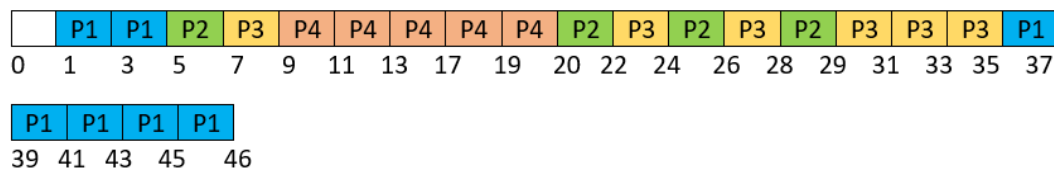
```

Figure 6: Kết quả testcase sched_1 có set priority

- Time slot 0: s0 được tải vào hệ thống, gán PID = 1, có priority = 4 (ưu tiên thấp nhất). CPU 0 bắt đầu xử lý PID 1.
- Time slot 1: s1 được tải vào hệ thống, PID = 2, với priority = 0 (cao hơn PID 1).
- Time slot 3: PID 1 đã hết time slice và được đưa vào hàng đợi. CPU 0 tiếp tục xử lý PID 1 (do chưa có process nào khác trong hàng đợi).
- Time slot 4: s1 được tải vào hệ thống với PID = 2 và priority = 0.
- Time slot 5: PID 1 bị tạm dừng và được đưa vào hàng đợi. CPU 0 bắt đầu xử lý PID 2 (vì PID 2 có priority = 0, cao hơn PID 1).
- Time slot 6: s2 được tải vào hệ thống với PID = 3 và priority = 0.
- Time slot 7: PID 2 đã hết time slice và được đưa vào hàng đợi. PID 3 được xử lý tiếp. s3 (PID = 4) được tải vào hệ thống và có priority = 0.
- Time slot 20: PID 4 hoàn tất và CPU 0 chuyển sang xử lý PID 2.
- Time slot 29: PID 2 hoàn tất và CPU 0 chuyển sang xử lý PID 3.
- Time slot 35: PID 3 hoàn tất và CPU 0 chuyển sang xử lý PID 1.
- Time slot 46: PID 1 hoàn tất và CPU 0 dừng lại.
- Hệ thống sử dụng một dạng Preemptive Scheduling: Tiến trình có priority cao hơn sẽ chiếm CPU ngay khi có sẵn tài nguyên. Các priority = 0 lần lượt được xử lý trong thời gian chúng đến mà không bị gián đoạn.

Output và biểu đồ Gantt:

CPU 0:



3 Memory Management

3.1 Cơ sở lý thuyết

3.1.1 Bộ nhớ ảo (The Virtual Memory Mapping in Each Process)

Virtual Memory Mapping: Trong mỗi tiến trình, không gian bộ nhớ ảo được tổ chức thành các vùng bộ nhớ liên tục (vm areas). Các vùng này có thể bao gồm đoạn mã, ngăn xếp hoặc heap. Tuy nhiên, trong bài tập lớn này, ta sẽ chỉ cho một process có một vùng nhớ duy nhất. Vùng nhớ có địa chỉ bắt đầu là *vm_area_start* và kết thúc ở *vm_area_end*. Mỗi vùng bộ nhớ được quản lý bởi một cấu trúc dữ liệu, bao gồm các thông tin về điểm bắt đầu và kết thúc của vùng, cùng với danh sách các vùng bộ nhớ trống. Không gian bộ nhớ ảo của mỗi tiến trình được chia thành nhiều vùng liên tục. Dù không gian này có thể trải dài toàn bộ địa chỉ ảo, nhưng vùng sử dụng thực tế bị giới hạn bởi một điểm gọi là *sbrk*. Trong khoảng giữa điểm bắt đầu của vùng và *sbrk*, có nhiều vùng bộ nhớ được quản lý và theo dõi bởi danh sách các vùng trống.

3.1.2 Bộ nhớ vật lý của hệ thống (The System Physical Memory)

Bộ nhớ vật lý của hệ thống: Bộ nhớ vật lý của hệ thống bao gồm RAM và SWAP. RAM là bộ nhớ chính, có thể truy cập trực tiếp từ CPU, trong khi SWAP là bộ nhớ phụ, được sử dụng để lưu trữ dữ liệu khi RAM đầy. Dữ liệu trong SWAP cần được chuyển vào RAM để CPU có thể truy cập. Bộ nhớ vật lý được quản lý thông qua các cấu trúc dữ liệu lưu trữ thông tin về frames. Các khung bộ nhớ (frames) này có thể được cấp phát hoặc giải phóng tùy theo nhu cầu của hệ thống. Việc sử dụng SWAP giúp hệ thống có thể mở rộng dung lượng bộ nhớ một cách hiệu quả, giảm thiểu tình trạng thiếu bộ nhớ khi RAM đầy. Đồng thời, việc quản lý bộ nhớ vật lý một cách hiệu quả giúp tối ưu hóa hiệu suất của hệ thống.

3.1.3 Sơ đồ dịch địa chỉ dựa trên phân trang

Khi thực hiện một instruction (VD: write 10 l 20). Ta sẽ lấy địa chỉ bắt đầu của thanh ghi 1 (*symrgtbl[1].start*) cộng với giá trị offset (20) để tạo thành địa chỉ logic.

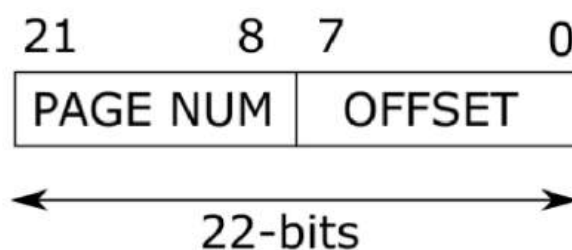
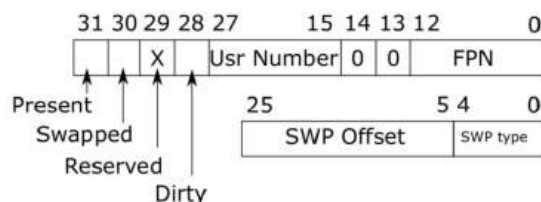


Figure 7: Địa chỉ logic

Từ địa chỉ logic có được, ta sẽ lấy ra được số trang và offset. Tra bảng page table với số trang đó, ta sẽ lấy ra được một page table entry, là một giá trị 32 bits chứa thông tin về frame number tương ứng và các thông tin khác (present or swapped). Từ frame number có được, ta sẽ tìm ra địa chỉ vật lý và giải giá trị vào đó.

Tuy nhiên, không phải lúc nào việc tra page table của process cũng cho ta một frame hợp lệ (frame hợp lệ là frame nằm trong RAM, do CPU chỉ có thể thao tác trực tiếp với bộ nhớ RAM). Khi frame tìm được từ bảng nằm trong SWAP, ta phải thực hiện giải thuật thay trang để thay



Hình 5: Các thành phần của một page table entry

Figure 8: Các thành phần của một page table entry

frame vào RAM đầu tiên ra SWAP và thay frame chúng ta cần vào RAM để truy xuất dữ liệu (giải thuật fifo).

3.1.4 Phân đoạn bộ nhớ nhiều phần (Multiple Memory Segments)

Multiple Memory Segments (phân đoạn bộ nhớ nhiều phần) là cách tổ chức bộ nhớ trong hệ điều hành nhằm chia không gian bộ nhớ của một tiến trình thành các phân đoạn khác nhau, mỗi phân đoạn phục vụ một chức năng cụ thể. Trong trường hợp này, bộ nhớ được chia thành hai phân đoạn chính:

- **Data Segment (Phân đoạn dữ liệu):** Data segment là một vùng nhớ được dành riêng cho việc lưu trữ dữ liệu tĩnh như các biến toàn cục, hằng số và các chuỗi ký tự cố định. Phân đoạn này thường được xác định trong quá trình biên dịch và không thay đổi kích thước trong suốt thời gian chạy của chương trình.

Quản lý tĩnh: Bộ nhớ được cấp phát cho data segment thường là tĩnh, tức là kích thước của nó không thay đổi trong quá trình chạy chương trình.

Địa chỉ liên tiếp: Data segment được tổ chức theo kiểu địa chỉ liên tiếp từ một vùng nhớ thấp và mở rộng lên phía trên, gần như được cố định từ khi chương trình được biên dịch.

- **Heap Segment (Phân đoạn heap):** là một vùng nhớ dành riêng cho việc cấp phát động. Dùng để lưu trữ dữ liệu động, chẳng hạn như các biến được cấp phát trong runtime qua các hàm như malloc, calloc và free trong C/C++. Kích thước của phân đoạn này có thể thay đổi trong thời gian chạy của chương trình, giúp các biến động có thể được cấp phát và giải phóng linh hoạt.

Quản lý động: Bộ nhớ trong heap segment được cấp phát động, tức là vùng nhớ có thể mở rộng hoặc thu hẹp tùy theo nhu cầu của chương trình.

Mở rộng theo nhiều hướng: Heap segment có thể mở rộng theo hai hướng:

- Go up (mở rộng lên): Bắt đầu từ một điểm cố định ở giữa không gian nhớ và mở rộng lên. Phương pháp này dễ quản lý nhưng có thể làm lãng phí bộ nhớ.
- Go down (mở rộng xuống): Bắt đầu từ địa chỉ tối đa của không gian nhớ và mở rộng xuống. Phương pháp này tận dụng tối đa bộ nhớ nhưng phức tạp hơn để quản lý.

3.1.5 Tóm tắt các triển khai theo định hướng phân trang

Quản lý bộ nhớ ảo (Virtual Memory Management): cho phép tiến trình sử dụng một không gian địa chỉ riêng biệt, không cần liên tiếp về mặt vật lý, mà được ánh xạ qua bảng trang.

Mục tiêu: Cách ly bộ nhớ giữa các tiến trình.

Sử dụng tài nguyên RAM hiệu quả bằng cách chỉ tải các trang cần thiết vào bộ nhớ.

Kỹ thuật phân trang (Paging) Bộ nhớ ảo được chia thành các trang (page) có kích thước cố định. Bộ nhớ vật lý (RAM) được chia thành các khung trang (frame) có cùng kích thước. Các trang ảo được ánh xạ tới các khung trang vật lý thông qua bảng trang (Page Table).

Địa chỉ ảo chia làm hai phần:

- Page Number (PN): Số thứ tự trang, dùng để tra cứu bảng trang.
- Page Offset (PO): Phần bù trong trang.

Bảng trang (Page Table) Chức năng: Lưu trữ ánh xạ từ số trang ảo sang khung trang vật lý. Theo dõi trạng thái của từng trang (trong RAM, SWAP, hay chưa được ánh xạ).

Dữ liệu mỗi mục trong bảng trang:

- Frame Number (F): Số khung trang vật lý.
- Bit trạng thái:
- Present: Trang có trong RAM hay không.
- Dirty: Trang đã được chỉnh sửa.

Heap "go down" vs "go up"

- Go up: Heap mở rộng từ giữa không gian nhớ lên trên. Quản lý đơn giản nhưng dễ lãng phí bộ nhớ.
- Go down: Heap mở rộng từ địa chỉ tối đa xuống dưới. Tận dụng toàn bộ bộ nhớ khả dụng, giảm phân mảnh.

Các Hằng Số Chính và Tác Động Của Chúng

- **MLQ_SCHED**: Bật tính năng lập lịch hàng đợi đa mức.
- **MAX_PRIO**: Xác định mức độ ưu tiên tối đa (140 trong trường hợp này).
- **MM_PAGING**: Kích hoạt module quản lý bộ nhớ trang.
- **MM_PAGING_HEAP_GODOWN**: Cho phép vùng nhớ heap mở rộng xuống dưới, bắt đầu từ địa chỉ bộ nhớ tối đa.
- **MM_FIXED_MEMSZ**: Tương thích ngược - duy trì kích thước RAM và SWAP cố định để hỗ trợ các cấu hình cũ. Hệ thống vẫn có thể hoạt động mà không cần thay đổi tệp input cũ.

3.2 Trả lời câu hỏi

Câu hỏi: Trong hệ điều hành đơn giản này, chúng ta triển khai thiết kế nhiều phân đoạn bộ nhớ hoặc vùng bộ nhớ trong khai báo mã nguồn. Ưu điểm của thiết kế nhiều phân đoạn được đề xuất là gì?

Việc thiết kế bộ nhớ ảo của process gồm nhiều vùng có tác dụng giúp ta quản lý bộ nhớ hiệu quả, cho phép cấp phát và giải phóng bộ nhớ một cách linh hoạt. Việc truy xuất dữ liệu cũng được đảm bảo độc lập giữa các vùng, bảo vệ dữ liệu và mã lệnh khỏi bị ghi đè hoặc truy cập trái

phép. Giúp tối ưu hóa đặc điểm sử dụng của các bộ nhớ và cũng hỗ trợ tốt cho việc chạy nhiều tiến trình cùng một lúc, giúp tránh xung đột và tăng hiệu suất hệ thống.

Câu hỏi: Điều gì sẽ xảy ra nếu chúng ta chia địa chỉ thành nhiều hơn 2 cấp trong hệ thống quản lý bộ nhớ phân trang?

Việc chia địa chỉ ảo (logical address) thành nhiều hơn 2 cấp độ sẽ làm tăng độ phức tạp của hệ thống quản lý bộ nhớ, đòi hỏi nhiều bước hơn để chuyển đổi từ địa chỉ ảo sang địa chỉ vật lý. Làm tăng chi phí truy cập bộ nhớ vì cứ mỗi lần truy cập sẽ phải trải qua nhiều cấp hơn. Với hệ thống 32-bit việc chia địa chỉ ảo thành bậc 2 là khả thi vì địa chỉ ảo có thể lưu được tuy nhiên với hệ thống lớn hơn, đánh dấu địa chỉ vùng nhớ sẽ không thể có đủ chỉ số để lưu trữ vùng nhớ. Thiết kế nhiều cấp giúp hệ thống quản lý bộ nhớ có khả năng mở rộng tốt hơn, hỗ trợ không gian địa chỉ lớn hơn mà không cần bảng trang quá lớn.

Câu hỏi: Ưu điểm và nhược điểm của phân đoạn bằng phân trang là gì?

Phân đoạn bằng phân trang (Segmentation with Paging) là một kỹ thuật kết hợp giữa hai phương pháp quản lý bộ nhớ: phân đoạn (segmentation) và phân trang (paging). Cả hai kỹ thuật này đều có ưu điểm và nhược điểm riêng khi áp dụng trong các hệ điều hành.

Ưu điểm của phân đoạn bằng phân trang

- Kết hợp ưu điểm của phân đoạn và phân trang:
 - Phân đoạn cho phép quản lý bộ nhớ dựa trên các khối dữ liệu có ý nghĩa (ví dụ: mã lệnh, dữ liệu toàn cục, ngăn xếp, heap), giúp quản lý bộ nhớ dễ hiểu và linh hoạt.
 - Phân trang giúp tối ưu hóa bộ nhớ vật lý bằng cách chia bộ nhớ thành các trang nhỏ, tránh tình trạng phân mảnh bộ nhớ ngoài (external fragmentation) và hỗ trợ việc chia sẻ bộ nhớ giữa các tiến trình hiệu quả.
- Quản lý bộ nhớ linh hoạt và hiệu quả:
 - Các phân đoạn có thể có kích thước khác nhau tùy thuộc vào từng loại dữ liệu, giúp tránh việc phân bổ bộ nhớ không đồng đều.
 - Paging giúp hệ thống phân chia bộ nhớ thành các khối kích thước đều, đảm bảo việc cấp phát bộ nhớ động được dễ dàng hơn.
- Tính hiệu quả trong việc phân chia bộ nhớ vật lý:
 - Paging giúp giảm thiểu external fragmentation bằng cách chia bộ nhớ thành các trang có kích thước cố định. Điều này cho phép bộ nhớ được cấp phát và giải phóng một cách hiệu quả.
 - Segmentation giúp giảm thiểu internal fragmentation (khi bộ nhớ được phân bổ không hết trong các khối cố định).
- Tăng cường chia sẻ bộ nhớ: Khi sử dụng phân trang, nhiều tiến trình có thể chia sẻ các trang chung mà không gặp vấn đề về bộ nhớ bị phân mảnh. Điều này đặc biệt hữu ích cho việc chia sẻ mã chương trình (code sharing), vì các tiến trình có thể dùng chung các trang mã lệnh mà không cần sao chép.
- Dễ dàng hoán đổi (swapping): Phân trang cho phép hoán đổi (swapping) bộ nhớ hiệu quả hơn vì các trang có kích thước nhỏ và đều, giúp việc đưa các trang vào bộ nhớ hoặc từ bộ nhớ ra dễ dàng và hiệu quả hơn.

Nhược điểm của phân đoạn bằng phân trang

- **Quản lý phức tạp:** Kết hợp cả phân đoạn và phân trang làm cho hệ thống trở nên phức tạp hơn, vì cần phải quản lý hai bảng:
Bảng phân đoạn để lưu thông tin về các phân đoạn, các trang trong từng phân đoạn.
Điều này làm tăng chi phí về bộ nhớ và thời gian khi cần phải tra cứu bảng phân đoạn và bảng trang mỗi khi cần truy xuất bộ nhớ.
- **Chi phí truy cập bộ nhớ cao hơn:** Do cần phải tra cứu hai bảng (bảng phân đoạn và bảng trang), việc truy cập bộ nhớ có thể trở nên chậm hơn so với các phương pháp phân bổ bộ nhớ khác (chẳng hạn như phân trang đơn giản).
- Hệ thống cần nhiều chỉ thị hơn trong mỗi phép toán truy cập bộ nhớ, do phải thực hiện tra cứu cả bảng phân đoạn và bảng trang.
- **Phân mảnh trong bảng phân trang:** Mặc dù phân trang giúp giảm phân mảnh bộ nhớ vật lý, nhưng vẫn có thể xảy ra phân mảnh bảng trang nếu các bảng này không được phân bổ hợp lý. Điều này có thể dẫn đến việc sử dụng bộ nhớ không hiệu quả.
- **Vấn đề phân bổ bộ nhớ cho bảng trang và bảng phân đoạn:** Cần cấp phát bộ nhớ cho cả bảng phân đoạn và bảng trang, điều này có thể dẫn đến việc sử dụng bộ nhớ không hiệu quả nếu các bảng này quá lớn, đặc biệt khi số lượng phân đoạn và số trang lớn.
- **Sự phức tạp trong hoán đổi (swapping):** Khi sử dụng phân trang, mỗi phân đoạn có thể có nhiều trang khác nhau. Khi hoán đổi, hệ thống phải quản lý và chuyển các trang của từng phân đoạn một cách độc lập, làm tăng độ phức tạp khi thực hiện hoán đổi giữa bộ nhớ vật lý và bộ nhớ phụ (swap space).

Câu hỏi: Tại sao dữ liệu lại được tách thành hai phân đoạn khác nhau: dữ liệu và heap? Vì việc phân chia bộ nhớ thành hai phân đoạn riêng biệt mang lại các lợi ích như sau:

- Để phân biệt cách sử dụng bộ nhớ:
 - Data segment: Phục vụ cho các biến tĩnh và toàn cục, không thay đổi trong runtime.
 - Heap segment: Phục vụ cho dữ liệu động, linh hoạt thay đổi trong runtime.
- Quản lý bộ nhớ dễ dàng hơn:
 - Data segment có thể được quản lý trực tiếp bởi trình biên dịch.
 - Heap segment được quản lý bởi các hàm runtime như malloc, free.
- Giảm thiểu xung đột giữa các loại dữ liệu: Các dữ liệu tĩnh và động được tách biệt để tránh việc ghi đè hoặc phá hủy dữ liệu lẫn nhau.
- Tăng cường bảo mật: Mỗi phân đoạn có thể được bảo vệ độc lập, giúp giảm thiểu lỗi ghi đè hoặc truy cập trái phép.
- Tối ưu hóa tài nguyên:
 - Data segment có kích thước cố định, tận dụng tốt không gian địa chỉ thấp.
 - Heap segment linh hoạt mở rộng, sử dụng hiệu quả không gian địa chỉ cao.
- Hỗ trợ phát hiện lỗi: Việc tách biệt các phân đoạn giúp dễ dàng phát hiện và xử lý lỗi (ví dụ như tràn bộ nhớ).

3.3 Hiện thực

3.3.1 Cấu trúc mã

Cấu trúc `vm_area_struct`

Chức năng: Quản lý các phân đoạn

```
1 struct vm_area_struct {  
2     unsigned long vm_id;  
3     unsigned long vm_start;  
4     unsigned long vm_end;  
5     unsigned long sbrk;  
6     struct vm_rg_struct *vm_freerg_list;  
7     struct vm_area_struct *vm_next;  
8 };
```

Cấu trúc `vm_rg_struct`

Chức năng: Theo dõi vùng bộ nhớ tự do:

```
1 struct vm_rg_struct {  
2     unsigned long rg_start;  
3     unsigned long rg_end;  
4     struct vm_rg_struct *rg_next;  
5 };
```

Cấu trúc `mm_struct`

Chức năng: Quản lý toàn bộ phân đoạn của tiến trình

```
1 struct mm_struct {  
2     struct vm_area_struct *mmap;  
3     struct vm_rg_struct symrgtbl[PAGING_MAX_SYMTBL_SZ];  
4 };
```

3.3.2 Các hàm cần triển khai

`mm-vm.c:`

- `__alloc`: Hàm dùng để cấp phát memory region trong một memory area. Hiện thực diễn ra như sau:
 - Tìm một region còn trống. Nếu có, dùng nó để cấp phát
 - Nếu không có, tức là area đã hết region còn trống. Ta sẽ tăng giới hạn của memory area, khi đó sẽ có region trống để cấp phát.
 - Trường hợp region trống đó lớn hơn kích cỡ cần cấp phát, ta sẽ chia region ban đầu làm 2, một dùng để cấp phát, một là region trống.

```
1     int __alloc(struct pcb_t *caller, int vmaid, int rgid, int  
2     size,  
3     int *alloc_addr) {  
4     /*Allocate at the topproof */  
5     struct vm_rg_struct rgnode;  
6     if (get_free_vmrg_area(caller, vmaid, size, &rgnode) == 0) {
```

```
7 caller->mm->symrgtbl[rgid].rg_start = rgnode.rg_start;
8 caller->mm->symrgtbl[rgid].rg_end = rgnode.rg_end;
9
10 *alloc_addr = rgnode.rg_start;
11
12 return 0;
13 }
14
```

- `__free`: Hàm dùng để thu hồi memory cấp phát. Ta chỉ đơn giản là thêm region đó vào danh sách region trống, và vô hiệu hoá region trong bảng `symrgtbl`.

```
1 int __free(struct pcb_t *caller, int vmaid, int rgid) {
2
3 if (rgid < 0 || rgid > PAGING_MAX_SYMTBL_SZ)
4 return -1;
5
6 /* TODO: Manage the collect freed region to freerg_list */
7 struct vm_rg_struct rgnode = {
8     .rg_start = caller->mm->symrgtbl[rgid].rg_start,
9     .rg_end = caller->mm->symrgtbl[rgid].rg_end,
10 };
11
12 // invalidate the region in symrgtbl
13 caller->mm->symrgtbl[rgid].rg_start = 0;
14 caller->mm->symrgtbl[rgid].rg_end = 0;
15 caller->mm->symrgtbl[rgid].rg_next = NULL;
16
17 /*enlist the obsoleted memory region */
18 enlist_vm_freerg_list(caller->mm, rgnode);
19
20 return 0;
21 }
22
```

- `validate_overlap_vm_area`: Kiểm tra xem memory area có đè lên memory area đã có hay không, tránh lỗi truy cập bộ nhớ. Ta đơn giản là lặp qua tất cả memory area để kiểm tra

```
1 int validate_overlap_vm_area(struct pcb_t *caller, int vmaid,
2 int vmastart,
3 int vmaend) {
4 struct vm_area_struct *vma = caller->mm->mmap;
5
6 /* TODO validate the planned memory area is not overlapped */
7
8 // there is a case where vma->vm_start == vma->vm_end
9 // we need to handle this case
10 while (vma != NULL) {
11     if (vma->vm_start == vma->vm_end) {
12         vma = vma->vm_next;
13         continue;
14     }
15     if ((vma->vm_start <= vmastart && vmastart <= vma->vm_end) ||
```

```
15     (vma->vm_start <= vmaend && vmaend <= vma->vm_end)) {  
16         return -1;  
17     }  
18     vma = vma->vm_next;  
19 }  
20  
21 return 0;  
22 }  
23
```

- `find_victim_page`: Hàm dùng trong giải thuật thay trang, khi ta đưa khung trang từ trong SWAP vào RAM, cũng có nghĩa là phải chọn khung trang để đưa từ RAM sang SWAP. Ta dùng giải thuật FIFO (first in first out) để thay trang.

```
1     int find_victim_page(struct mm_struct *mm, int *retpgn) {  
2         struct pgn_t *pg = mm->fifo_pgn;  
3  
4         /* TODO: Implement the theoretical mechanism to find the victim page */  
5         // we implement page replacement algorithm here  
6         // we use FIFO algorithm  
7  
8         if (pg == NULL) {  
9             *retpgn = -1;  
10            return -1;  
11        }  
12  
13        *retpgn = pg->pgn;  
14        mm->fifo_pgn = pg->pg_next;  
15        free(pg);  
16        return 0;  
17    }  
18
```

- `pg_getpage`: Hàm để lấy khung trang từ số trang. Hiện thực diễn ra như sau:
 - Từ số trang, ta lấy được entry trong bảng trang
 - Từ entry ta kiểm tra xem khung trang có trong bộ nhớ không. Nếu có ta đơn giản lấy khung trang đó và trả về.
 - Nếu không ta sẽ dùng giải thuật thay trang để hoán đổi khung trang cần sử dụng với khung trang nào đó dựa trên giải thuật thay trang của chúng ta
 - Trả về khung trang cần tìm

```
1     int pg_getpage(struct mm_struct *mm, int pgn, int *fpgn,  
2     struct pcb_t *caller) {  
3         uint32_t pte = mm->pgd[pgn];  
4  
5         if (!PAGING_PAGE_PRESENT(  
6             pte)) { /* Page is not online, make it actively living */  
7             int vicpgn, swpfpn;  
8             int vicfpn;
```

```
8      uint32_t vicpte;
9
10     int tgtfpn = PAGING_SWP(pte); // the target frame storing our
    variable
11
12     /* TODO: Play with your paging theory here */
13     /* Find victim page */
14     if (find_victim_page(caller->mm, &vicpgn) == -1) {
15         perror("find_victim_page failed, maybe no page allocated\
n");
16         return -1;
17     }
18     vicpte = caller->mm->pgd[vicpgn];
19     vicfpn = PAGING_FPN(vicpte);
20
21     /* Get free frame in MEMSWP */
22     MEMPHY_get_freefp(caller->active_mswp, &swfpn);
23
24     /* Do swap frame from MEMRAM to MEMSWP and vice versa*/
25     /* Copy victim frame to swap */
26     __swap_cp_page(caller->mram, vicfpn, caller->active_mswp, swfpn)
    ;
27     /* Copy target frame from swap to mem */
28     __swap_cp_page(caller->active_mswp, tgtfpn, caller->mram, vicfpn)
    ;
29
30     /* Update page table */
31     pte_set_swap(&mm->pgd[vicpgn], 0, swfpn);
32
33     /* Update its online status of the target page */
34     // pte_set_fpn() & mm->pgd[pgn];
35     pte_set_fpn(&mm->pgd[pgn], vicfpn);
36
37     enlist_pgn_node(&caller->mm->fifo_pgn, pgn);
38
39     MEMPHY_put_freefp(caller->active_mswp, tgtfpn);
40 }
41
42 *fpn = PAGING_FPN(pte);
43
44 return 0;
45 }
46
```

mm.c:

- `vmap_page_range`: Hàm dùng để ánh xạ các trang bắt đầu từ một địa chỉ nhất định trong bảng trang đến các khung trang. Ta sẽ lặp qua các entry trong bảng bắt đầu từ `addr` và ánh xạ nó đến khung trang được cho trước theo thứ tự.

```
1      int vmap_page_range(
2      struct pcb_t *caller,    // process call
3      int addr,                // start address which is aligned to pagesz
```

```

4  int pgnum,                // num of mapping page
5  struct framephy_struct *frames, // list of the mapped frames
6  struct vm_rg_struct *ret_rg) // return mapped region, the real
   mapped fp
7  {
8      // no guarantee all given pages are mapped
9      // uint32_t * pte = malloc(sizeof(uint32_t));
10     // struct framephy_struct *fpit = malloc(sizeof(struct
11     framephy_struct));
12     // int fpn;
13     int pgit = 0;
14     int pgn = PAGING_PGN(addr);
15
16     ret_rg->rg_end = ret_rg->rg_start =
17     addr; // at least the very first space is usable
18
19     // fpit->fp_next = frames;
20
21     /* TODO map range of frame to address space
22     * [addr to addr + pgnum*PAGING_PAGESZ
23     * in page table caller->mm->pgd[]
24     */
25     struct framephy_struct *fpit = frames;
26     int ret_val = 0;
27     for (pgit = 0; pgit < pgnum; pgit++) {
28         if (fpit == NULL) {
29             // out of frames while still have pages to map
30             ret_val = -1;
31             break;
32         }
33         int fpn = fpit->fpn;
34         uint32_t *pte = &caller->mm->pgd[pgn + pgit];
35         pte_set_fpn(pte, fpn);
36         fpit = fpit->fp_next;
37
38         /* Tracking for later page replacement activities (if needed)
39         * Enqueue new usage page */
40         enlist_pgn_node(&caller->mm->fifo_pgn, pgn + pgit);
41     }
42
43     ret_rg->rg_end = addr + pgit * PAGING_PAGESZ;
44
45     return ret_val;
46 }

```

mm-memphy.c:

- MEMPHY_dump: dùng để dump memory (in thông tin bộ nhớ) để theo dõi thông tin bộ nhớ vật lý. Ta sẽ in mỗi hàng là 4 byte trong bộ nhớ vật lý tại mỗi địa chỉ (là một con số chia hết cho 4).

```

1  int MEMPHY_dump(struct memphy_struct *mp) {
2  /*TODO dump memphy contnt mp->storage

```

```
3      *      for tracing the memory content
4      */
5      printf("---MEM DUMP---\n");
6      uint32_t* word_storage = (uint32_t*)mp->storage;
7      int i;
8      for (i = 0; i < mp->maxsz / 4; i++)
9          if (word_storage[i] != 0)
10             printf("%08x: %08x\n", i * 4, word_storage[i]);
11
12     return 0;
13 }
14
```

3.3.3 Kết quả thực thi

Tạo ra 2 testcase mới tên là test_mem và test_swap để kiểm tra hoạt động của bộ nhớ và bộ nhớ khi xuất hiện trường hợp bộ nhớ RAM hết dung lượng và phải thực hiện swap.

```
2 1 1
1048576 16777216 0 0 0
0 m0s 0
1 m1s 1

2 1 1
256 16777216 0 0 0
0 m0s 0
1 m1s 1
```

Phân tích kết quả:

- Time slot 0: Một tiến trình được tải từ file m0s với PID = 1 và PRIO = 0. Đây là tiến trình có độ ưu tiên cao nhất (nếu hệ thống dùng MLQ hoặc một thuật toán ưu tiên khác).
- Time slot 1: CPU 0: Dispatched process 1: CPU 0 bắt đầu thực thi tiến trình PID = 1.
- "Dispatch" nghĩa là tiến trình được chọn từ hàng đợi (run queue) để chạy trên CPU.
- Time slot 2 - 3: Không có thao tác cụ thể ngoài việc CPU xử lý tiến trình (có thể xử lý logic của tiến trình hoặc thực hiện các lệnh alloc, malloc, hoặc write).
- Time slot 3: CPU 0: Put process 1 to run queue: CPU 0 tạm thời dừng tiến trình PID = 1 và đưa nó trở lại hàng đợi chạy (run queue). CPU 0: Dispatched process 1: Ngay sau đó, tiến trình PID = 1 được lấy ra khỏi hàng đợi để tiếp tục chạy. Điều này thường xảy ra nếu thuật toán CPU chia sẻ thời gian (time-sharing) hoặc khi tiến trình yêu cầu tài nguyên khác.
- Quá trình này lặp lại tại Time slot 5 và Time slot 7.
- write region=1 offset=20 value=102: Tiến trình PID = 1 ghi giá trị 102 vào vùng nhớ 1, tại offset = 20.

```

Time slot 0
ld routine
Loaded a process at input/proc/m0s, PID: 1 PRIO: 0
Time slot 1
CPU 0: Dispatched process 1
Time slot 2
Time slot 3
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
Time slot 4
Time slot 5
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
write region=1 offset=20 value=102
print_pgtbl: 0 - 512
00000000: 80000001
00000004: 80000000
---MEM DUMP---
Time slot 6
write region=2 offset=1000 value=1
print_pgtbl: 0 - 512
00000000: 80000001
00000004: 80000000
---MEM DUMP---
Time slot 7
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
write region=0 offset=0 value=0
print_pgtbl: 0 - 512
00000000: 80000001
00000004: c0000000
---MEM DUMP---
00000014: 00000066
000000b0: 00000001
Time slot 8
CPU 0: Processed 1 has finished
CPU 0 stopped

```

Figure 9: Kết quả testcase_mem

- print_pgtbl: Hiển thị bảng trang (Page Table) từ vị trí 0 đến 512.
- 00000000: 80000001: Đây là định dạng của bảng trang, trong đó:
- Bit 31 (presented) = 1: Trang đang được ánh xạ trong RAM.
- Bit 30 (swapped) = 0: Trang không bị swapped ra ngoài.
- Bit 0-12: Chứa thông tin khung trang (Frame Number).
- 80000000: Trang này chưa được ánh xạ (bit 31 = 0).
- MEM DUMP: Hiển thị trạng thái bộ nhớ sau khi ghi, nhưng không có thay đổi nào được ghi nhận.
- write region=2 offset=1000 value=1: Tiến trình PID = 1 ghi giá trị 1 vào vùng nhớ 2, tại offset = 1000.
- print_pgtbl: Cấu trúc bảng trang không thay đổi so với lần trước.
- MEM DUMP:
- 00000014: 00000066: Có dữ liệu tại địa chỉ này, nhưng không rõ nguyên nhân thay đổi từ hành động trước đó.
- write region=0 offset=0 value=0: Tiến trình PID = 1 ghi giá trị 0 vào vùng nhớ 0, tại offset = 0.
- print_pgtbl: 00000004: c0000000: Địa chỉ thứ hai đã được cập nhật với Bit 30 = 1 (swapped). Điều này cho thấy một số trang đã bị swap từ RAM sang SWAP (quá trình swapping).

- MEM DUMP:
- 000000b0: 00000001: Bộ nhớ vật lý đã ghi giá trị 1 vào địa chỉ này.
- Time slot 8: CPU 0: Processed 1 has finished: Tiến trình PID = 1 đã hoàn thành nhiệm vụ và thoát. CPU 0 stopped: Không còn tiến trình nào trong hàng đợi để thực thi.

```
Time slot 0
ld_routine
Loaded a process at input/proc/m0s, PID: 1 PRIO: 0
CPU 0: Dispatched process 1
Time slot 1
Time slot 2
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
Time slot 3
Time slot 4
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
write region=1 offset=20 value=102
print_pgtbl: 0 - 512
00000000: 00000000
00000004: 00000000
---MEM DUMP---
Time slot 5
write region=2 offset=1000 value=1
print_pgtbl: 0 - 512
00000000: 00000000
00000004: 00000000

---MEM DUMP---
Time slot 6
CPU 0: Put process 1 to run queue
CPU 0: Dispatched process 1
write region=0 offset=0 value=0
print_pgtbl: 0 - 512
00000000: 00000000
00000004: 00000000
---MEM DUMP---
Time slot 7
CPU 0: Processed 1 has finished
CPU 0 stopped
```

Figure 10: Kết quả testcase_swap

Phân tích kết quả:

- Tiến trình m0s được tải với PID = 1, độ ưu tiên 0 (cao)
- Tiến trình được dispatch lên CPU 0.
- Các time slots (1-6) cho thấy tiến trình liên tục được đưa vào và ra khỏi hàng đợi (run queue) và được dispatch để tiếp tục thực thi.
- write region=1, offset=20, value=102: Ghi vào bộ nhớ tại vùng 1, nhưng bảng trang không thay đổi.
- write region=2, offset=1000, value=1: Ghi vào bộ nhớ tại vùng 2, nhưng bảng trang vẫn không thay đổi.
- write region=0, offset=0, value=0: Ghi vào bộ nhớ tại vùng 0, không có thay đổi trong bảng trang và bộ nhớ.
- Time slot 7: Tiến trình hoàn thành và CPU dừng vì không còn tiến trình nào để thực thi.

4 Put It All Together

4.1 Cơ sở lý thuyết

Tổng kết lại các phần ở trên, ta được một hệ điều hành đơn giản như sau:

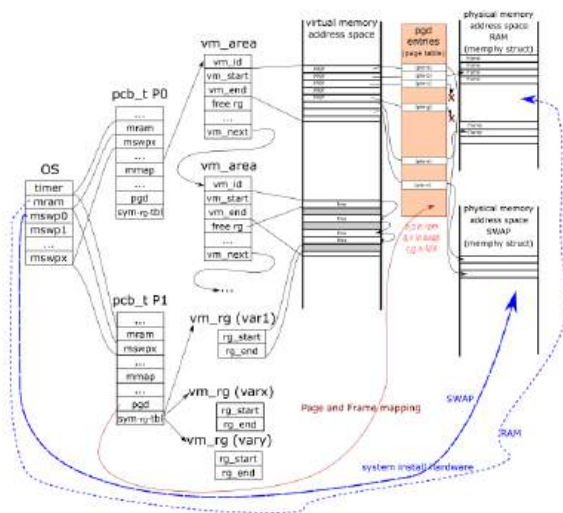


Figure 11: Tổ chức đầy đủ của một hệ điều hành đơn giản

4.2 Trả lời câu hỏi

Câu hỏi: Điều gì sẽ xảy ra nếu quá trình đồng bộ hóa không được xử lý trong hệ điều hành đơn giản của bạn? Minh họa vấn đề của hệ điều hành đơn giản của bạn (đầu ra gần) bằng ví dụ nếu có. **Trả lời:** Do hệ điều hành sử dụng nhiều CPU, việc không xử lý đồng bộ sẽ xảy ra một số vấn đề đồng bộ phổ biến:

- Data race: Xảy ra khi các process hoặc thread cùng thực hiện đọc và ghi lên một khối dữ liệu, sẽ dẫn đến tính không nhất quán của dữ liệu.
- Race condition: Trường hợp thứ tự sử dụng của nhiều process hoặc thread có thể dẫn đến kết quả không như mong muốn.
- Deadlock: Xảy ra khi nhiều luồng đều chờ lẫn nhau để truy cập tài nguyên. Việc này sẽ khiến process hoặc thread bị ngưng vô thời hạn.

Ví dụ: Có hai tiến trình A và B đọc và ghi cùng một vùng bộ nhớ đồng thời, dữ liệu có thể bị ghi đè lên nhau dẫn đến mất mát hoặc không nhất quán. Cụ thể, hai tiến trình A và B đang cố gắng cập nhật giá trị của một biến được chia sẻ 'count' đồng thời. Trường hợp không có đồng bộ hóa:

- Tiến trình A đọc giá trị 'count = 5'
- Tiến trình B đọc giá trị 'count = 5'
- Tiến trình A tăng 'count' lên 6 và ghi lại



- Tiến trình B cũng tăng 'count' lên 6 và ghi lại

Kết quả cuối cùng là 'count = 6' thay vì giá trị mong muốn là 7, do hai tiến trình đã ghi đè lên nhau. Điều này có thể dẫn đến lỗi logic nghiêm trọng và khó phát hiện trong ứng dụng.